PowerDAG: Reliable Agentic AI System for Automating Distribution Grid Analysis

Emmanuel O. Badmus, Amritanshu Pandey

arXiv:2603.17418v3 [eess.SY] 22 Apr 2026

***Abstract*—This paper introduces PowerDAG, an agentic AI system for automating complex distribution-grid analysis. We address the reliability challenges of state-of-the-art agentic systems in automating complex engineering workflows by introducing two innovative active mechanisms: (i) adaptive retrieval, which uses a similarity-decay cutoff algorithm to dynamically select the most relevant annotated exemplars as context, and (ii) just-in-time (JIT) supervision, which actively intercepts and corrects tool-usage violations during execution. On a benchmark of unseen distribution grid analysis queries, PowerDAG achieves a 100% success rate with GPT-5.2 and 94.4–96.7% with smaller open-source models, outperforming base ReAct (41–88%), LangChain (30–90%), and CrewAI (9–41%) baselines by margins of 6–50 percentage points.**

***Index Terms*—agentic AI systems, distribution grid analysis, long-prompt context, relevant context, workflow orchestration**

I. Introduction

The rapid growth of distributed energy resources (DERs), including rooftop solar photovoltaics, battery storage, and electric vehicles, necessitates frequent and complex distribution system analyses [1]. These analyses support operations and planning through simulation tools (e.g., power flow for voltage profiles [2], [3]), optimization tools (e.g., dynamic hosting capacity to quantify solar curtailment [4]), and data analysis [5]. Each analysis demands multi-step execution: model assembly, tool orchestration and execution, and result validation. Many stakeholders who require these analyses, particularly smaller utilities, regulators, and policy groups, lack the engineering capacity to conduct them at scale, thereby delaying interconnection approvals and limiting grid modernization [6].

The central challenge is executing multi-step tool sequences across well-established tools (e.g., GridLabD [7]) and data pipelines (e.g., advanced metering infrastructure (AMI) database) while maintaining correct tool selection, argument bindings, and environment state [8]. While rule-based engines can automate standardized study templates, they are inflexible, demand ongoing manual curation, and cannot robustly accommodate the diversity of real-world analyses [9]. LLMs offer a promising alternative, enabling users to describe desired analyses in natural language and receive a corresponding orchestration of tool calls [10].

Early approaches provide LLMs with tool descriptions and prompt them to orchestrate tool-call sequences for engineers to execute [11], [12], but this still requires manual execution and debugging [13]. Agentic systems eliminate manual intervention by proposing tools and arguments based on the evolving interaction history: the user query, tool descriptions, actions, and observations from prior steps. However, LLMs struggle to generate valid execution plans for specialized domains due to limited exposure to domain-specific procedural knowledge during pretraining [11]. Two main strategies address this: supervised fine-tuning (SFT) adapts model weights using domain-specific training data [12], while long in-context learning (ICL) provides task-relevant contents in the prompt without weight updates [14], [15]. SFT effectively internalizes domain patterns but requires substantial compute, domain-specific training data, and complete retraining when tools or data pipelines evolve.

ICL avoids these costs by augmenting prompts with task-relevant context [14]. However, ICL effectiveness depends substantially on the quality and structure of the given context [15]. General instructional sources (papers, manuals) describe concepts but rarely specify the tool's execution logic and reasoning, as each typically covers only one tool or method and ignores the rest of the pipeline. As a result, no single source specifies cross-tool interactions, data handoffs, or state updates. For example, separate GridLAB-D and AMI database guides exist, but neither defines the *cross-tool* orchestration to map AMI data into consistent feeder loads.

To close these gaps, recent works [16], [17] demonstrate promising results by collating *expert* experiences of running complex analyses through annotated exemplars. These exemplars structurally codify actionable procedural knowledge, including orchestration of tool calls, state dependencies, and argument structures. However, in this paradigm, selecting annotated exemplars is critical because LLMs have a finite context window, which limits the number of exemplars we can include in the prompt. Also, irrelevant annotated exemplars introduce distractor tool traces that dilute the relevant procedural signal, degrading tool selection under long contexts while increasing prompt length and token cost [18]. To counter these gaps, recent work [16] uses RAG-style similarity-based retrieval to select the top-k exemplars per query.

The problem: Static top-k retrieval in [16] has two primary drawbacks. First, the choice of k introduces inherent trade-offs: small k omits latent tool dependencies for complex analysis queries, while large k includes irrelevant exemplars that degrade tool selection and inflate context size [18]. Second, in its current form, [16] isolates relevant annotated exemplars based on query similarity. While this produces a semantic match between the current query and selected exemplars, it may discard exemplars with relevant tool dependencies and orchestration.

Another critical challenge arises during tool execution. For unseen queries, the agent invokes an LLM to recommend tool calls given relevant annotated exemplars. The agent executes these tool calls and uses the return message as the primary signal of correctness. This approach is brittle in stateful environments, where tools often execute successfully on stale or uninitialized objects without raising exceptions. For instance, if an agent runs a *power flow solver* but bypasses a prerequisite *state update* call, the subsequent plotting function will be executed but produce a wrong plot. The agent misinterprets this *silent failure* as success and proceeds. To prevent such irreversible state corruption and silent failures, a proactive supervisor must enforce dependency constraints.

Proposed Solution: To address these fundamental gaps, we implement a two-pronged strategy, within our agentic system PowerDAG:

- **Adaptive Retrieval:** We introduce a similarity-decay cutoff

Emmanuel O. Badmus and Amritanshu Pandey are with the Department of Electrical and Biomedical Engineering, University of Vermont, Burlington, VT, USA. Email: emmanuel.badmus@uvm.edu, amritanshu.pandey@uvm.edu

algorithm to select a subset of annotated exemplars via query text and tool-call traces to isolate procedurally relevant context.
- **Just-in-time Supervision:** We introduce a pre-invocation guardrail that verifies state dependencies, preventing corruption by blocking invalid calls and returning one-shot corrective advisories.

II. Related Works

Tool-augmented agents with LLM supervision are increasingly central to orchestrating complex interactions between tools and data for automating analyses.

A. LLMs for Tool Orchestration

Commercial LLM chat interfaces (e.g., ChatGPT, Claude, Gemini) concatenate user queries with historical conversation context and route them to their backend LLM (e.g., GPT-5.2, Claude Opus 4.5, Gemini 3 Pro) for response generation. In a standard chat interface, the underlying LLM cannot run external distribution system simulators (e.g., ChatGPT cannot run OpenDSS on its own). It therefore returns guidance or code that the user can execute and validate outside the interface. Power-system applications include OpenDSS file generation [19], iterative script refinement [20], and result visualization [21]. An alternative approach augments prompts with tool descriptions and requests complete tool-call sequences with bound arguments [11], [12]. These require manual tool calls. As an improvement, agentic frameworks employ an iterative execution loop in which the LLM proposes a single tool call, observes the result, and conditions subsequent decisions on feedback [10], [13]. Agentic grid applications include GridMind for OPF automation [22], GridAgent for contingency analysis [23], RePower for solver-driven planning [24], and X-GridAgent for executable multi-step workflows across power-system simulators [25]. However, these systems rely on solver-in-the-loop checks but do not implicitly enforce prerequisite and cross-tool state dependencies, thereby limiting reliability in stateful pipelines.

B. Exemplar-Conditioned Orchestration

Agentic frameworks automate tool orchestration but require procedural guidance to select tools, sequence calls, and satisfy prerequisite dependencies. Recent systems, therefore, condition decisions on retrieved exemplars of successful tool use, rather than relying only on natural-language manuals [15], [26]. Tool-retrieval methods also use iterative feedback from LLM calls to refine which tools and exemplars to use [27]. Several agent systems store and retrieve prior trajectories as procedural memory to improve future decisions under long horizons [28]. Other work constructs reusable multi-step traces from interaction and reuses them via retrieval and refinement across tasks [29], [30]. GeoFlow explicitly names these traces *workflows* and represents them as Activity-on-Vertex graphs with step-level tool objectives, improving task success and reducing token use [17]. [16] conditions distribution grid analysis agents on annotated exemplars of verified execution traces and reports higher Pass@1 than unstructured retrieval baselines in its experiments.

C. Retrieval of Relevant Exemplars

One can reduce the overall context in a prompt by isolating relevant context. Retrieval-Augmented Generation (RAG) uses embedding similarity to retrieve external knowledge, and conditions the model's generation on this retrieved context [26]. Many systems implement dense retrieval by indexing exemplar candidates in a vector store and selecting the top-k nearest neighbors under cosine similarity [31]–[33]. Some deployments filter exemplar candidates using a minimum similarity threshold, either alone or combined with top-k, to reduce low-relevance context [34], [35]. For procedural exemplars that codify expert experience and record executable multi-step tool use, such as call ordering, argument patterns, and intermediate-state handoffs, fixed top-k retrieval creates a coverage-noise trade-off.

[18], [36]. This issue is particularly acute for tool-calling and execution agents, where retrieved content must encode executable structure—call ordering, argument patterns, and prerequisite dependencies—rather than narrative descriptions [16].

D. Runtime Supervision of Tool Execution

Another key challenge arises when tools execute on residual or incorrect states without raising exceptions, leading to silent failures. These are primarily due to violations of tool dependencies, causing them to operate on stale or uninitialized environment state. As a lightweight mitigation, long-context prompting techniques can incorporate dependencies or adversarial examples that violate dependencies and state constraints to steer tool selection and execution [37], [38], but sampling remains probabilistic and cannot guarantee satisfaction of dependencies or constraints. Post-hoc self-correction methods revise actions in response to detected failures [39], but they do not prevent silent errors when a tool executes without error on stale or uninitialized objects. Several systems implement runtime supervision by interposing deterministic guardrails between the agentic orchestrator and the executor to check preconditions before each call and to return targeted feedback on violations. For instance, [40] represents each tool as a state-transition rule with explicit entry-state requirements and expected state updates, then uses symbolic checks to validate that the transition is admissible. Also, [41] simulates candidate actions to predict violations before execution. Lastly, [42] defines event-triggered safeguard policies that gate high-stakes actions (e.g., requiring user confirmation for an unverified transfer recipient) before execution. However, they check single-step safety and do not target long-horizon workflow correctness under exemplar retrieval.

III. Preliminaries

A. Agentic AI System

An *agentic AI system* couples an agent and LLM with a stateful environment in a closed loop. We model the agent as an *orchestrator* that maps the interaction history with relevant context to the next tool-call action; the environment as an *executor* which executes the call and returns an observation. Within a closed loop, at each iteration k, the agent orchestrates and issues a tool-call action a_k, and the tools are executed within the stateful environment in its current state to produce the next state and observation.

$$\texttt{Agent} \xrightarrow{a_k} \texttt{Environment} \xrightarrow{o_k} \texttt{Agent} \xrightarrow{a_{k+1}} \cdots$$

B. State Transition and Tool Classification

The environment maintains a state at iteration k, $s_k \in \mathcal{S}$. Each tool $t \in \mathcal{T}$ defines a state-observation map $F_t : \mathcal{S} \times \Phi_t \to \mathcal{S} \times \mathcal{O}$, where Φ_t is the argument space of tool t and $\mathcal{O}$ is the observation space returned to the agent. We classify tools by whether they modify the environment state: (i) a *Write* tool updates persistent objects stored in the registry or simulator, so $s_{k+1} \neq s_k$, (ii) a *Read* tool queries the current objects and returns an observation, while maintaining environment states $s_{k+1} = s_k$. A *Read* tool becomes valid only after the required *Write* tools have constructed the required state, which imposes an execution order captured by the *workflow*, modeled with a directed acyclic graph (DAG). We label a tool as *Write* iff it mutates any persistent environment object; otherwise it is *Read*, and we validate this with state-print tests.

C. Workflow as a Directed Acyclic Graph (DAG)

An agent responds to a query q_u by recommending a set of tool invocations subject to state-dependent dependencies. We formalize this workflow as a directed acyclic graph (DAG), $\mathcal{G}_w = (V_w, E_w)$. The vertex set $V_w = \{v_1, \ldots, v_m\}$ represents distinct tool invocations, where each $v_i = t_i(\phi_i)$ consists of a tool $t_i \in \mathcal{T}$ and its arguments ϕ_i. The edge set $E_w \subset V_w \times V_w$ encodes execution constraints: a directed edge $(v_i, v_j) \in E_w$ implies a strict ordering, ensuring v_i completes before v_j begins. For example, two invocations with the same parent and no dependency path between them can execute in either order (e.g., $t_4(\phi_4)$ and $t_5(\phi_5)$ in Fig. 1). Also, a tool can execute only after all of its incoming dependencies complete (e.g., $t_6(\phi_6)$ depends on both $t_4(\phi_4)$ and $t_5(\phi_5)$). Acyclicity rules out circular prerequisites and guarantees the existence of a topological order.

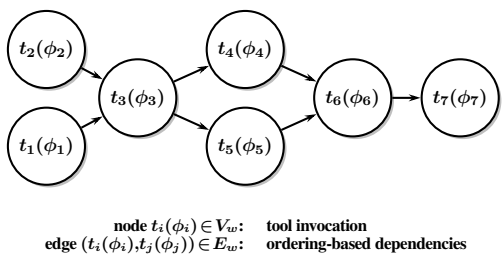

Fig. 1. Workflow as a directed acyclic graph (DAG). Nodes denote tool invocations; directed edges encode ordering-based dependencies.

D. Annotated Query-Workflow Pair Exemplars

Power systems experts, including researchers and engineers, solve complex distribution-grid analysis queries by following established tool sequences. We encode this experience as annotated query-workflow pair exemplars. Each record stores $(q^{(i)}, w^{(i)})$, where $q^{(i)}$ is a natural-language query and $w^{(i)}$ is the executed workflow that produces the required outputs in the stateful environment.

IV. An Agentic AI System: PowerDAG

Informal Problem:

- ***Given*** an *unseen* distribution grid analysis query q_u in natural language, a tool set $\mathcal{T}$ and associated data pipelines, a stateful environment, and annotated exemplars of expert experiences $\mathcal{W}_{\text{av}}$,

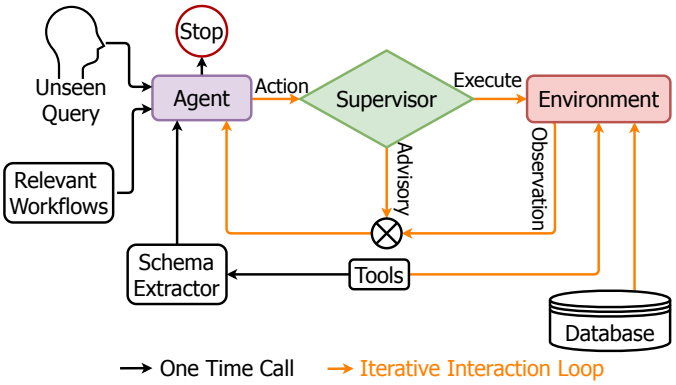

Fig. 2. PowerDAG execution architecture. Initialization: the schema extractor builds Σ from the tool set, and a retrieval module selects relevant $\mathcal{W}_{sub}$ from $\mathcal{W}_{av}$ for the unseen query q_u. Iterative loop: the agent proposes an action, the supervisor either blocks and returns an advisory or executes the action. If executed, an observation returns. The loop stops when the agent gives a textual final response.

- ***Goal*** is to return accurate analysis outputs for the query together with an executable workflow w that generates them.

PowerDAG is an agentic-AI system, which automates complex distribution-grid analysis. We introduce PowerDAG architecture with two innovations (i) adaptive exemplar retriever and (ii) just-in-time prerequisite supervision, which enables PowerDAG to surpass the state-of-the-art in terms of correctness and performance.

A. PowerDAG System Architecture

Let $\mathcal{T}$ denote the tool set and let Σ denote the corresponding set of tool schemas. For each tool $t \in \mathcal{T}$, we define a schema $\sigma_t = \langle n_t, d_t, \Pi_t \rangle$, where n_t is the tool name, d_t is a short description, and Π_t is the argument signature, i.e., the list of input arguments and their data types. An invocation of tool t uses concrete values ϕ_t for these parameters.

$$\Sigma := \{\sigma_t\}_{t \in \mathcal{T}} \tag{1}$$

We also maintain an archive $(\mathcal{W}_{\text{av}})$ of expert-generated annotated query-workflow pair exemplars records. For a query q_u, an adaptive retriever selects a subset $\mathcal{W}_{\text{sub}} \subset \mathcal{W}_{\text{av}}$ and appends $\mathcal{W}_{\text{sub}}$ to the prompt alongside q_u , interaction history H, and Σ. The system then runs the closed-loop interaction between the agent and the environment, as shown in Fig. 2. At iteration k, the agent orchestrates an action:

$$a_k \sim \pi_\theta(\cdot | q_u, \mathcal{W}_{\text{sub}}, H_k, \Sigma) \tag{2}$$

The agent either terminates and returns a final text response, or it returns a tool-call action $a_k = \langle n_k, \phi_k \rangle$.

In PowerDAG, we place a deterministic just-in-time supervisor between the agent and the environment (see Fig. 2), so the supervisor checks the proposed tool-call action against prerequisite rules $\mathcal{C}$ before execution. If a rule is violated, the supervisor blocks the call and returns the advisory observation $o_k = \alpha(n_k)$ to the agent; no tool is executed, and the environment state remains unchanged. Otherwise, the executor executes $t_k(\phi_k)$ in the environment to produce new state and observation (s_{k+1}, o_k), after which the history updates as $H_{k+1} = H_k \oplus (a_k, o_k)$. Next, we describe key innovations of PowerDAG: an *adaptive retrieval algorithm*

that selects a subset $\mathcal{W}_{\text{sub}}$ from $\mathcal{W}_{\text{av}}$ for query conditioning, and a *just-in-time supervision algorithm* that validates tool-call preconditions and returns advisory feedback before execution.

B. Adaptive Retrieval of Annotated Exemplars

Our goal is to develop an adaptive method to select annotated exemplars $\mathcal{W}_{\text{sub}}$ from the expert archive $\mathcal{W}_{\text{av}}$ that overcomes the drawbacks of works that choose top-k exemplars, where k is a fixed hyper-parameter [16]. To that end, we develop (i) an adaptive cutoff computed from the ranked similarity profile and (ii) a second-stage filter that keeps only exemplars whose workflows are structurally consistent with q_u,

The expert archive is $\mathcal{W}_{\text{av}} = \{(q_i, w_i)\}_{i=1}^{N}$, where each record pairs a stored query q_i with its expert annotated workflow w_i. The context selector forms a text-similarity ranking in three steps: it encodes the unseen query q_u and each stored query q_i into embeddings $\psi(.)$, and computes cosine similarities between embeddings. It then sorts the scores and re-indexes the archive-annotated exemplars in W_{av}, $i \to n$, by descending similarity score, such that:

$$\text{Sim}_i = \cos(\psi(q_u), \psi(q_i)), \quad i = 1, \ldots, N \tag{3}$$

$$\text{Sim}_1 \geq \text{Sim}_2 \geq \cdots \geq \text{Sim}_N \tag{4}$$

The context selector treats $\{\text{Sim}_i\}_{i=1}^{N}$ as a discrete relevance profile in descending order (i.e., most relevant exemplar appears first and the least appears last) and selects an adaptive cutoff (given by index i_{cutoff} in ordered set) by fitting a two-segment least-squares model to the sequence and choosing the breakpoint that minimizes the weighted fit error:

$$i_{\text{cutoff}} = \arg\min_{2 \leq m \leq N-2} \left[\frac{m}{N} \Phi(1, m) + \frac{N-m}{N} \Phi(m+1, N) \right] \tag{5}$$

Here $\Phi(a, b)$ denotes the RMSE of the least-squares affine fit of (n, Sim_n) over $n = a, \ldots, b$; Appendix C gives the full derivation of this objective. This yields the candidate set:

$$\mathcal{W}_{\text{cand}} = \{(q_n, w_n)\}_{n=1}^{i_{\text{cutoff}}} \tag{6}$$

Text-similarity ranking alone is insufficient because it can retrieve candidates whose query text resembles q_u while their associated workflows do not match the tool structure implied by q_u. Conditioning on such mismatched exemplars can mislead the agent and reduce correctness. At retrieval time, the correct workflow for q_u is unknown, so we cannot directly verify procedural alignment. We therefore apply an LLM-based filter that scores each candidate using both its query text and its expert-verified workflow trace, and retains only those that are procedurally consistent with q_u. The LLM acts only as a gate over a small candidate set; it selects among existing expert traces and does not synthesize a new tool sequence. This bounded discrimination task is simpler than long-horizon workflow generation. The filter runs once at initialization and returns the final subset $\mathcal{W}_{\text{sub}}$:

$$J = \Omega\left(\pi_\theta^{\text{filter}}(\cdot \mid q_u, \mathcal{W}_{\text{cand}})\right), \quad \mathcal{W}_{\text{sub}} := \{(q_{(j)}, w_{(j)}) : j \in J\} \tag{7}$$

where Ω is a deterministic parser that converts the LLM output into an index set J. Appendix D lists the exact filtering prompt. Fig. 3 illustrates the two-stage selection used to construct $\mathcal{W}_{\text{sub}}$.

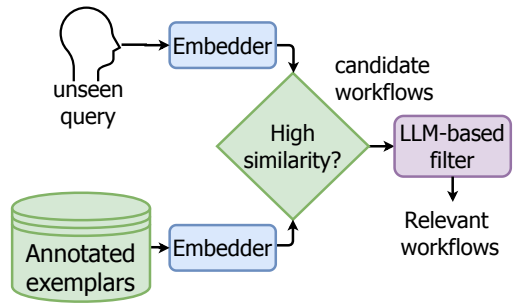

Fig. 3. Two-stage exemplar selection. The system embeds q_u and $\{q_i\}_{i=1}^{N}$, ranks by cosine similarity, selects an adaptive cutoff i_{cutoff}, and applies an LLM-based workflow filter to produce the final subset $\mathcal{W}_{\text{sub}}$.

C. Just-In-Time Supervision

Even with access to relevant exemplars $\mathcal{W}_{\text{sub}}$, the agent can propose a tool call whose required prerequisite (or dependencies) *Write* tools have not executed, so the call acts on stale state and may erroneously overwrite stored objects necessary in later steps, creating a compounding error problem. We address this failure mode by inserting a deterministic supervisor that checks prerequisite conditions before each tool execution using an explicit rule library $\mathcal{C}$ that encodes a conservative subset of must-run-before dependencies. We build $\mathcal{C}$ from (i) tool documentation and implementation inspection that identifies which state objects each tool reads and writes, and (ii) learning repeated prerequisite patterns in the annotated query-workflow pair exemplars $\mathcal{W}_{\text{av}}$.

Let $\mathcal{N}$ denote the set of tool names and let $\text{dom}(\mathcal{C}) \subseteq \mathcal{N}$ denote the subset covered by prerequisite rules. For each $n \in \text{dom}(\mathcal{C})$, the rule specifies a prerequisite set $\mathcal{N}^{\text{req}}(n) \subseteq \mathcal{N}_{\text{write}}$ and an advisory message $\alpha(n)$. If $n_k \notin \text{dom}(\mathcal{C})$, the supervisor applies no prerequisite check and allows execution.

Formally, given a proposed tool invocation $a_k = \langle n_k, \phi_k \rangle$, the supervisor flags a violation if the execution history fails to satisfy the prerequisite set for n_k. In such cases, the supervisor intercepts the action, blocks execution, and returns an advisory that explicitly lists the missing *Write* tools. We implement this advisory via the prompt in Appendix E, which inserts the violation notice and the blocked tool call action name. Conversely, if no violation is detected, the supervisor authorizes the tool for execution.

1) Prerequisite Constraint Specification

Building on the *Read/Write* tool classification, we encode prerequisite knowledge as an explicit rule library $\mathcal{C}$ indexed by tool name. Let $\mathcal{N}$ denote the universe of tool names and let $\mathcal{N}_{\text{write}} \subseteq \mathcal{N}$ denote the *Write*-tool set. For each tool name $n \in \mathcal{N}$ covered by the supervisor, the corresponding rule specifies (i) the prerequisite *Write* tools that must have executed earlier and (ii) a fixed advisory message returned when a prerequisite is missing. We represent this rule as

$$c(n) = \langle n, \mathcal{N}^{\text{req}}(n), \alpha(n) \rangle \tag{8}$$

where $\mathcal{N}^{\text{req}}(n) \subseteq \mathcal{N}_{\text{write}}$ is the required prerequisite set and $\alpha(n)$ is the advisory message returned when $\mathcal{N}^{\text{req}}(n)$ is not satisfied. If a tool name n has no entry in $\mathcal{C}$, the supervisor applies no prerequisite check to calls of that tool.

2) Prerequisite Check and One-Shot Advisory

Given the constraint set $\mathcal{C}$, the supervisor enforces prerequisites at runtime, issuing a one-shot advisory upon detecting a violation.

Specifically, at any step k, when the agent proposes a tool-call action $a_k = \langle n_k, \boldsymbol{\phi}_k \rangle$, the supervisor evaluates it against the history of executed tools, $\mathcal{N}_k^{\text{exec}} \subseteq \mathcal{N}$ (accumulated over steps $0,\ldots,k-1$). If n_k is governed by a rule in $\mathcal{C}$, the supervisor verifies that its dependencies are satisfied by checking if $\mathcal{N}^{\text{req}}(n_k) \subseteq \mathcal{N}_k^{\text{exec}}$. A violation is flagged when this inclusion condition fails with an indicator function:

$$\upsilon_k := \mathbf{1}[\mathcal{N}^{\text{req}}(n_k) \nsubseteq \mathcal{N}_k^{\text{exec}}] \tag{9}$$

Equivalently, $\upsilon_k = 1$ if and only if the action requires at least one prerequisite tool that has not yet been executed. If $\upsilon_k = 1$, the supervisor blocks the proposed call and returns the advisory observation $o_k = \alpha(n_k)$. To prevent infinite loops, we enforce a one-shot policy where subsequent repetition of the call is permitted. Otherwise, the supervisor permits execution. This behavior defines a deterministic update rule for the returned observation o_k:

$$o_k = \begin{cases} \alpha(n_k) & \text{if } \upsilon_k = 1 \\ \mathcal{E}(t_k(\boldsymbol{\phi}_k)) & \text{otherwise} \end{cases} \tag{10}$$

Algorithm 1 describes the end-to-end execution of PowerDAG.

Algorithm 1 PowerDAG Algorithm

Require: query q_u, tool set $\mathcal{T}$ with its schemas Σ, annotated exemplars $\mathcal{W}_{\text{av}}$, prerequisite rules $\mathcal{C}$, stateful environment.
Ensure: executed workflow trace w and final response

Phase 1: Adaptive Retrieval
  Compute similarities $\{\text{Sim}_i\}_{i=1}^{N}$ and **rank and re-index** ▷ *Eqs.* (3),(4)
  Calculate cutoff i_{cutoff}, $\mathcal{W}_{\text{cand}} \leftarrow \{(q_n, w_n)\}_{n=1}^{i_{\text{cutoff}}}$ ▷ *Eq.* (5),(6)
  Filter $J \leftarrow \Omega(\pi_\theta^{\text{filter}}(\cdot \mid q_u, \mathcal{W}_{\text{cand}}))$, $\mathcal{W}_{\text{sub}} \leftarrow \{(q_j, w_j) : j \in J\}$ ▷ *Eq.* (7)
Phase 2: Just-in-time supervision
Init $s_0 \leftarrow \emptyset$, $H_0 \leftarrow ()$, $w \leftarrow ()$, $\mathcal{N}_0^{\text{exec}} \leftarrow \emptyset$, $k \leftarrow 0$
loop
  Action $a_k \sim \pi_\theta(\cdot \mid q_u, \mathcal{W}_{\text{sub}}, H_k, \Sigma)$ ▷ *Eq.* (2)
  if a_k is not final **then**
    Extract $\langle n_k, \boldsymbol{\phi}_k \rangle$ from a_k
    if $n_k \notin \text{dom}(\mathcal{C})$ **then** ▷ *no prerequisite rule*
      Execute & Observe $(s_{k+1}, o_k) \leftarrow \mathcal{E}(s_k, t_k(\boldsymbol{\phi}_k))$
      Update workflow $w \leftarrow w \oplus a_k$
      $\mathcal{N}_{k+1}^{\text{exec}} \leftarrow \mathcal{N}_k^{\text{exec}} \cup \{n_k\}$
    else
      Set $\upsilon_k \leftarrow \mathbf{1}[\mathcal{N}^{\text{req}}(n_k) \nsubseteq \mathcal{N}_k^{\text{exec}}]$ ▷ *Eq.* (9)
      if $\upsilon_k = 1$ **then**
        Observation (advisory) $o_k \leftarrow \alpha(n_k)$ ▷ *Eq.* (10)
        Freeze state: $s_{k+1} \leftarrow s_k$ ▷ *no environment state change*
        $\mathcal{N}_{k+1}^{\text{exec}} \leftarrow \mathcal{N}_k^{\text{exec}}$
      else
        Execute & Observe $(s_{k+1}, o_k) \leftarrow \mathcal{E}(s_k, t_k(\boldsymbol{\phi}_k))$
        Update workflow $w \leftarrow w \oplus a_k$
        $\mathcal{N}_{k+1}^{\text{exec}} \leftarrow \mathcal{N}_k^{\text{exec}} \cup \{n_k\}$
      end if
    end if
    Append history $H_{k+1} \leftarrow H_k \oplus (a_k, o_k)$
    $k \leftarrow k+1$
  else
    return w, final response
  end if
end loop

V. Experimental Setup

This section defines the queries, baselines, models, and scoring protocol used to measure PowerDAG correctness and efficiency.

A. Queries and Study Scope

We evaluate PowerDAG on 30 expert-curated distribution-grid queries (Appendix A). The query set spans four analysis classes: (i) *data-only query*, (ii) *unbalanced three-phase power flow* [43], (iii) *dynamic hosting capacity* with $\ell_1/\ell_2/\ell_\infty$ formulations [44]–[47], and (iv) *three-phase infeasibility analysis* with ℓ_1- and ℓ_2-objective variants [48], [49]. The queries vary in workflow length from ≤ 3 tool calls to > 10 tool calls.

B. Curation of Annotated Query-Workflow Pair Exemplars

We curate 50 annotated query-workflow pair exemplars for in-context learning. Each pair includes an expert-written query along with a fully verified workflow. All 50 annotated query-workflow pair exemplars are distinct from the 30 evaluation queries in Section V-A. 10 of the 50 annotated query-workflow pair exemplars cover the same four classes of analyses as the 30 unseen queries. To evaluate the retrieval accuracy of PowerDAG, the remaining 40 annotated query-workflow pair exemplars represent analyses different from four classes of analyses in 30 unseen queries. Code and data are available at https://github.com/PowerDAG/PowerDAG

C. Tool Sets, Data Interfaces, and Agentic Systems

Table I summarizes the systems compared that we use in our evaluation, along with the available features. To ensure a controlled comparison, we keep the environment, data pipelines, and tool sets fixed across all agentic systems. The full tool set contains 82 tools; the 30 evaluation queries require only 21 tools spanning data loading, solver runs, constraint checks, plotting, and export. We intentionally added 61 tools during evaluation to test whether agents select the correct tools within an expanded action space. We run all experiments on real-world distribution feeders (we do not name in this version due to double-blind review). Data pipelines include the AMI database, GeoJSON GIS files, time-series PV generation data, and network data in GridLAB-D format corresponding to the same real-world utility.

TABLE I
Comparison of Baseline Systems and Enabled Components

Component	Vanilla ReAct [13]	LangChain [50]	CrewAI [51]	PowerChain [16]	PowerDAG
Tool Orchestration	✓	✓	✓	✓	✓
Action-observation loop	✓	✓	✓	✓	✓
ICL Exemplars	✗	✗	✗	✓	✓
Retrieval	✗	✗	✗	✓	✓
Adaptive Retrieval	✗	✗	✗	✗	✓
JIT Supervisor	✗	✗	✗	✗	✓

D. LLMs and Compute Platforms

We evaluate two proprietary LLMs, GPT-4o-mini [52] and GPT-5.2 [53], via the OpenAI API [54]. We also evaluate four open-weight LLMs: Qwen3-14B [55], Qwen3-32B [56], gpt-oss-20b [57], and gpt-oss-120b [57]. We serve the open-weight models with the `vLLM` server [58] on a node with 2× NVIDIA H100 SXM GPUs (80 GB each) [59]. For exemplar selection, we embed queries and archived exemplar texts using OpenAI text embedder text-embedding-3-large [60].

E. Evaluation and Metrics

We evaluate the correctness of workflows produced by PowerDAG and the baseline agentic systems on each unseen evaluation query using *pass@k* and *Precision*. For each query and method, the agent generates $k \in \{1,3\}$ independent workflows because LLM decoding is non-deterministic, so tool-call traces can vary across trials [61]. Pass@k then measures whether at least one of these k workflows yields the same numeric result as the expert ground-truth workflow when executed in the same environment; we report pass@1 and pass@3. Precision measures whether the agent's executed tool-call trace is equivalent to the expert ground-truth workflow. We also report an efficiency metric, Tokens per pass@1, which measures the average token cost per successful run. Appendix F provides the full metric definitions and equations, including the description of evaluator we wrote to compute these metrics.

VI. Results

This section reports benchmark results for PowerDAG on the unseen distribution-grid analysis queries. Table II compares PowerDAG against four baselines (CrewAI, LangChain, Vanilla ReAct, and PowerChain [16]) across six LLMs. PowerDAG achieves the highest Pass@1 and Pass@3 rates across all models. It also uses fewer tokens per successful run. The improvements are largest on smaller models (GPT-4o-mini, GPT-OSS-20B, Qwen3-14B), where other agents fail to complete multi-step workflows. We analyze these results in four parts: baseline comparisons, model scaling behavior, efficiency analysis, and ablation studies.

TABLE II
COMPARISON OF AGENTIC SYSTEMS ACROSS METRICS

Method	GPT-4o-mini				GPT-5.2			
	P@1	P@3	Pr.	Tk/P@1(K)	P@1	P@3	Pr.	Tk/P@1(K)
CrewAI	41.11	43.33	28.89	920.17	34.44	43.33	17.78	719.17
LangChain	43.33	46.67	4.44	111.40	90.00	93.33	71.11	58.61
Vanilla ReAct	41.11	46.67	0.00	16.28	87.78	93.33	81.11	7.72
PowerChain	76.67	83.33	60.00	11.04	96.67	96.67	96.67	11.59
PowerDAG	**94.44**	**96.67**	**77.78**	8.98	**100.00**	**100.00**	**100.00**	8.64
	GPT-OSS-20B				GPT-OSS-120B			
CrewAI	17.78	33.33	16.67	238.76	30.00	36.67	27.78	116.80
LangChain	32.22	33.33	18.89	107.18	44.44	46.67	35.56	147.59
Vanilla ReAct	56.67	66.67	43.33	12.99	86.67	90.00	75.56	8.36
PowerChain	86.67	90.00	83.33	10.49	84.44	86.67	75.56	13.82
PowerDAG	**96.67**	**100.00**	**92.22**	9.48	**94.44**	**96.67**	**90.00**	9.81
	Qwen3-14B				Qwen3-32B			
CrewAI	15.56	26.67	12.22	290.02	35.56	36.67	34.44	52.30
LangChain	30.00	36.67	5.56	171.75	36.67	50.00	15.56	200.51
Vanilla ReAct	46.67	56.67	33.33	28.86	60.00	73.33	52.22	21.97
PowerChain	82.22	93.33	76.67	19.20	85.56	93.33	81.11	18.50
PowerDAG	**95.56**	**100.00**	**88.89**	16.59	**96.67**	**100.00**	**93.33**	18.44

Note: P@1: Pass@1; **P@3**: Pass@3; **Pr.**: Precision; **Tk/P@1**: Tokens per Pass@1. P@1, P@3, and Precision are in %. Token counts are reported in thousands ($k = 10^3$).

A. Performance Comparison against Baselines

Table II shows that PowerDAG outperforms all baselines on every model. On GPT-5.2, PowerDAG achieves perfect scores: 100% Pass@1, 100% Pass@3, and 100% Precision. PowerChain reaches 96.67% on all three metrics. LangChain, an open-source agentic workflow orchestrator, achieves 90% Pass@1, while Vanilla ReAct and CrewAI perform worse. On GPT-OSS-120B, PowerDAG maintains 94.44% Pass@1 and 90.0% Precision. PowerChain reaches 84.44% Pass@1 and Vanilla ReAct 86.67%, though Vanilla ReAct has lower Precision (75.56%). LangChain and CrewAI show further degradation. Fig. 4 visualizes these results by combining Pass@1 and Precision into a single quality score for each system.

B. Robustness on Smaller Models

PowerDAG maintains high performance on smaller models, achieving 94%-97% Pass@1 across GPT-4o-mini, GPT-OSS-20B, Qwen3-14B, and Qwen3-32B. Standard baselines struggle: Vanilla ReAct drops to 41%-60% Pass@1, while CrewAI and LangChain fall below 50%. Compared to PowerChain, PowerDAG improves Pass@1 by 10-18 percentage points and Precision by 9-18 percentage points. The gains are largest on GPT-4o-mini, PowerDAG reaches 94.44% Pass@1 while PowerChain achieves 76.67%. These results show that PowerDAG makes smaller, more cost-effective models viable for complex grid analysis tasks.

C. Efficiency and Token Economy

PowerDAG uses fewer tokens per successful run than all baselines in most settings (Table II). On GPT-4o-mini, PowerDAG requires 9.0k tokens per Pass@1 compared to 16.3k for Vanilla ReAct. LangChain and CrewAI are far less efficient at 111.4k and 920.2k, respectively. Two factors explain these gains. First, PowerDAG makes fewer mistakes. Higher Pass@1 means fewer retries and failed trajectories. While PowerDAG adds context tokens by including retrieved exemplars, this overhead pays off: PowerDAG completes more workflows correctly on the first attempt. Second, PowerDAG avoids prompt bloat. LangChain and CrewAI include extensive tool descriptions and role-based context in every call. This overhead pushes too much information, and the models lose attention [18].

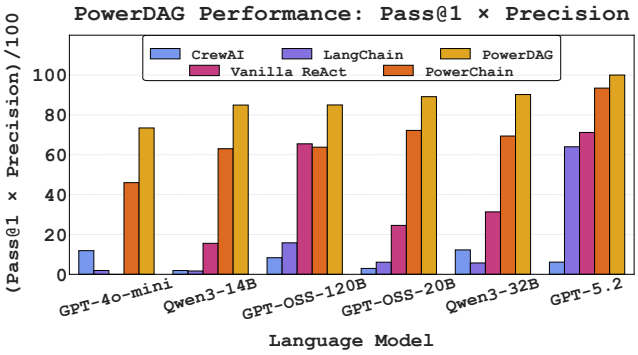

Fig. 4. **Combined performance score (Pass@1 × Precision) across models.** This metric captures both first-attempt success and workflow correctness. PowerDAG achieves the highest scores on all six models, with near-perfect performance on four of them. The gap between PowerDAG and baselines is largest on smaller models.

D. End-to-End Impact of the Proposed Extensions

We compare PowerDAG with PowerChain [16] to quantify the end-to-end effect of the two extensions introduced in this paper: adaptive retrieval and just-in-time supervision. Table II reports Pass@1, Pass@3, and Precision for both systems across the evaluated models. The largest gains occur on smaller models. For example, on GPT-4o-mini, Pass@1 increases from 76.7% to 94.4% and Precision rises from 60.0% to 77.8%. On Qwen3-14B, Pass@1 increases from 82.2% to 95.6% and Precision from 76.7% to 88.9%. PowerDAG attains 100% Pass@3 on four models where PowerChain falls short.

These results are consistent with the intended role of *Adaptive Retrieval* and *Just-in-Time Supervision*, but this comparison does not isolate each extension's marginal contribution.

VII. Conclusion

PowerDAG automates distribution-grid analysis by orchestrating and executing tool-call sequences in a stateful environment. PowerDAG improves reliability with the following key contributions:

- *Correctness:* Adaptive context retrieval with just-in-time supervision help achieve near 100% accuracy for most models.
- *Model efficiency:* Higher first-attempt success reduces the need for iterative corrections, significantly lowering total token consumption and runtime.
- *Democratization:* High accuracy with open-source models shows PowerDAG can avoid costly commercial models.

Appendix

AI Usage Disclosure: We used generative AI to revise grammar and improve writing. We reviewed each suggestion and take responsibility for the final content.

Appendix A
Benchmark Task Set

We show representative samples from the 30 benchmark tasks used for PowerDAG evaluation. Each task is phrased as a natural-language query that the agent must translate into executable workflows.

Easy

1. How many capacitors are in the Rochester feeder, Vermont?

⋮

10. Can you export all nodes whose voltage is 120 volts in the South Hero feeder into a txt file?

Medium

1. Without running any analysis, plot the bus voltage magnitudes from the input data for the Glover distribution network at 11:00 AM, 18th of March, 2025.

⋮

20. For the Glover network at 6:00 PM on March 21, 2025, run L1 current infeasibility analysis. Set the convergence tolerance to 1e-6. Report the top 5 buses with the highest infeasible currents. Then, plot the voltage magnitude after this analysis.

Hard

1. Load network file for the Rochester feeder in Vermont on 2025-03-19 at 15:00. Run dynamic hosting capacity. Assume solar irradiance data for that date and time. Choose a sparse curtailment strategy. Enforce the voltage limit between 0.95 and 1.05 at all nodes. Limit transformer flow to 110% of rated. After solving the problem, plot the curtailed power as a feeder topology map, with each bus color-coded by its curtailed power value and edges drawn according to the network connectivity.

⋮

30. Conduct a L2 norm current infeasibility study for the Stowe feeder, representing conditions at 10:00 on March 23, 2025. In this analysis, a global voltage of 0.94–1.06 p.u. is applied to all the buses except buses with IDs of 10, 25, and 36. These buses' voltage should be between 0.98 and 1.02 p.u. Additionally, enforce a global transformer loading limit of 115% of its rating. Solve the model using IPOPT, with a tolerance of 1e-6 and a maximum iteration count of 1000. After solving the problem, generate a plot visualizing the infeasible currents of each bus on a map, and another plot showing the bus voltage on a map.

Appendix B
Annotated Verified Workflow-Task Pairs

This appendix presents the 10 related annotated, verified workflow–query pairs (Annotated Query-Workflow Pair Exemplars) used as expert exemplars during in-context learning and evaluation. Each pair links a natural-language task to a verified domain workflow.

Annotated Query-Workflow Pair Exemplars

1. Load the{location} feeder model and report counts of buses, lines, and transformers as a quick integrity check.

⋮

10. For the{location} feeder at{YYYY-MM-DD}{HH:MM}, run L2 current-infeasibility with transformer limits at 110%, applying global voltage bounds of 0.90–1.10 p.u. and custom bounds of 0.95–1.05 p.u. for 5th, 10th, and 30th buses by ID. Then plot both the bus voltages and infeasible currents.

Full benchmark task set and complete 50 annotated query-workflow pair exemplars repository are available at: https://github.com/PowerDAG/PowerDAG We note that while this work is limited to ten representative queries, ongoing collaborations with VEC and other partners aim to scale the benchmark to hundreds of expert-level tasks. We also plan to open-source additional datasets and agent baselines to support reproducible power-grid analytics research.
We note that while this work is limited to ten representative queries, ongoing collaborations with utility partners aim to scale the benchmark to hundreds of expert-level tasks. We also plan to open-source additional datasets and agent baselines to support reproducible power-grid analytics research.

Appendix C
Derivation of the Adaptive Cutoff Objective

This appendix defines Φ and derives the weighted two-segment objective used in (5). Assume $\{\mathrm{Sim}_n\}_{n=1}^{N}$ is sorted in non-increasing order as in (4). For a candidate breakpoint $m \in \{2,\ldots,N-2\}$, define the left and right index sets:

$$I_L(m) := \{1,\ldots,m\}, \qquad I_R(m) := \{m+1,\ldots,N\} \tag{C.1}$$

Least-squares line on an index interval. For any interval $a{:}b$ with $1 \le a < b \le N$, define the best affine fit of (n,Sim_n) by:

$$(\hat{\alpha}_{a:b},\hat{\beta}_{a:b}) := \arg\min_{\alpha,\beta\in\mathbb{R}} \sum_{n=a}^{b} \Big(\mathrm{Sim}_n - (\alpha+\beta n)\Big)^2 \tag{C.2}$$

Segment Error $\Phi(a,\,b)$. We define $\Phi(a,\,b)$ as the RMSE of the least-squares affine fit to $\{(n,\mathrm{Sim}_n)\}_{n=a}^{b}$:

$$\Phi(a,b) = \sqrt{\frac{1}{b-a+1}\sum_{n=a}^{b}\Big(\mathrm{Sim}_n - (\hat{\alpha}_{a:b}+\hat{\beta}_{a:b}n)\Big)^2} \tag{C.3}$$

Weighted two-segment objective. For breakpoint m, the two segment RMSE values are combined as:

$$\begin{aligned}\mathcal{E}(m) &:= \frac{|I_L(m)|}{N}\Phi(1,m) + \frac{|I_R(m)|}{N}\Phi(m+1,N)\\ &= \frac{m}{N}\Phi(1,m) + \frac{N-m}{N}\Phi(m+1,N)\end{aligned} \tag{C.4}$$

Appendix D
Workflow-Filter Prompt

We filter the candidate exemplar set W_{cand} with the following prompt. When agent starts, {query}is replaced by q and {candidates}.

Workflow Filter Prompt

```
Select the best out of these candidate workflows to keep
as in-context exemplars.
USER QUERY: {query}
CANDIDATES (JSON): {candidates}
Each candidate has: - "query" - "workflow" (tool name +
arguments)
Keep a candidate if its workflow helps solve the user
query. Exclude only if clearly unrelated.
Return ONLY a JSON list of indices to keep (no duplicates).
Example: [0, 2, 5]
```

Appendix E
Supervision Advisory Prompt

When the supervisor blocks a tool call, an advisory for the blocked action {tool_name}is sent as observation back to the agent.

Advisory Injection Prompt

```
DECISION REQUIRED: {violation_msg}
Carefully consider the user's query. You must now make
ONE of these choices: 1. If the prerequisite IS needed
for this query: Call the suggested tool(s) first 2. If the
prerequisite is NOT needed: Retry your original call to
{tool_name} now
Make a tool call. Do not respond with text.
```

Appendix F
Evaluation Metrics

A. Correctness Metrics

Correctness evaluates how well an agent-orchestrated workflow reproduces the expert workflow's outcome and tool-call procedure. We assess correctness along two axes: (i) *success*, which checks that the agent's workflow covers all required expert tool calls with compatible arguments and dependency order while allowing state-changing extra calls, and (ii) *trace equivalence*, which checks whether the agent's tool-call sequence matches the expert sequence.

Pass@k. Pass@k estimates the probability that at least one of the k sampled workflows is successful on a query [61]. Let n be the number of attempted runs and let s be the number of runs that satisfy the success criterion. We compute Pass@k as:

$$\text{Pass@k} = 1 - \frac{\binom{n-s}{k}}{\binom{n}{k}}, \quad 1 \le k \le n \tag{F.1}$$

Higher values of Pass@k indicate that the model is more likely to produce a successful workflow within k independent samples.

Precision (Pr). Precision measures trace-level agreement by testing whether the agent's executed workflow is equivalent to the expert workflow after converting both tool-call traces to dependency DAGs. For run i, let $\hat{w}_i$ be the agent-executed workflow and let w_e be the expert workflow; we set $\text{Pr}_i = 1$ if $\hat{w}_i \equiv w_e$ and $\text{Pr}_i = 0$ otherwise:

$$\text{Pr} = \frac{1}{n}\sum_{i=1}^{n} \text{Pr}_i = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\hat{w}_i \equiv w_e] \tag{F.2}$$

B. Efficiency Metrics

Efficiency measures the inference resources required to obtain successful workflows; in this study, we use total token count (prompt + completion) as a model-agnostic proxy for that cost. We report it as a single efficiency metric, Tokens per Pass@1 (Tk/P@1), defined as the average tokens consumed per successful run:

$$\text{Token/Pass@1} = \frac{\sum_{i=1}^{n} T_i}{n \cdot \text{Pass@1}}, \quad \text{only when Pass@1} > 0 \tag{F.3}$$

C. Evaluator Implementation

We developed an evaluator to compute Pass@k and Precision. The evaluator represents each workflow as a dependency DAG, so it can capture parallel tool calls. Independent calls can be swapped without changing the environment state or outputs, so the evaluator does not penalize such reorderings. The implementation is available at https://github.com/PowerDAG/PowerDAG.

References

[1] J. Bank and B. Mather, "Analysis of the impacts of distribution connected pv using high-speed datasets," in *2013 IEEE Green Technologies Conference (GreenTech)*. IEEE, 2013, pp. 153–159.

[2] C. S. Cheng and D. Shirmohammadi, "A three-phase power flow method for real-time distribution system analysis," *IEEE Transactions on Power systems*, vol. 10, no. 2, pp. 671–679, 2002.

[3] W. H. Kersting, "Distribution system modeling and analysis," in *Electric power generation, transmission, and distribution*. CRC press, 2018, pp. 26–1.

[4] A. K. Jain *et al.*, "Dynamic hosting capacity analysis for distributed photovoltaic resources—framework and case study," *Applied Energy*, vol. 280, p. 115633, 2020.

[5] R. Singh *et al.*, "Dms industry survey," Argonne National Laboratory, Tech. Rep. ANL/ESD-17/11, Apr. 2017. [Online]. Available: https://publications.anl.gov/anlpubs/2017/06/136567.pdf

[6] 21st Century Energy Workforce Advisory Board, "Opportunities for american workers in energy," U.S. Department of Energy, Tech. Rep., jul 2025. [Online]. Available: https://www.energy.gov/sites/default/files/2025-07/EWAB_Special_Report_Opportunities_for_American_Workers_in_Energy.pdf

[7] D. P. Chassin *et al.*, "Gridlab-d: an agent-based simulation framework for smart grids," *Journal of Applied Mathematics*, vol. 2014, no. 1, p. 492320, 2014.

[8] Electric Power Research Institute (EPRI), "Distribution modeling guidelines: Executive summary—recommendations for system and asset modeling for distributed energy resource assessments," Electric Power Research Institute, Palo Alto, CA, Tech. Rep. 3002008894, aug 2016.

[9] B. G. Buchanan and E. H. Shortliffe, *Rule based expert systems: the mycin experiments of the stanford heuristic programming project (the Addison-Wesley series in artificial intelligence)*. Addison-Wesley Longman Publishing Co., Inc., 1984.

[10] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.

[11] S. G. Patil *et al.*, "Gorilla: Large language model connected with massive apis," *Advances in Neural Information Processing Systems*, vol. 37, pp. 126 544–126 565, 2024.

[12] Y. Qin *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," *arXiv preprint arXiv:2307.16789*, 2023.

[13] S. Yao *et al.*, "React: Synergizing reasoning and acting in language models," in *The eleventh international conference on learning representations*, 2022.

[14] T. Brown *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[15] W. M. I.-C. L. Work, "Rethinking the role of demonstrations: What makes in-context learning work?"

[16] E. O. Badmus *et al.*, "Powerchain: A verifiable agentic ai system for automating distribution grid analyses," *arXiv preprint arXiv:2508.17094*, 2025.

[17] A. Bhattaram *et al.*, "Geoflow: Agentic workflow automation for geospatial tasks," in *Proceedings of the 33rd ACM International Conference on Advances in Geographic Information Systems*, 2025, pp. 1150–1153.

[18] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.

[19] R. S. Bonadia *et al.*, "On the potential of chatgpt to generate distribution systems for load flow studies using opendss," *IEEE Transactions on Power Systems*, vol. 38, no. 6, pp. 5965–5968, 2023.

[20] M. Jia *et al.*, "Enhancing llms for power system simulations: A feedback-driven multi-agent framework," *IEEE Transactions on Smart Grid*, 2025.

[21] S. Jin and S. Abhyankar, "Chatgrid: Power grid visualization empowered by a large language model," in *2024 IEEE Workshop on Energy Data Visualization (EnergyVis)*. IEEE, 2024, pp. 12–17.

[22] H. Jin *et al.*, "Gridmind: Llms-powered agents for power system analysis and operations," in *Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 560–568.

[23] Y. Zhang *et al.*, "Grid-agent: An llm-powered multi-agent system for power grid control," *arXiv preprint arXiv:2508.05702*, 2025.

[24] Y.-X. Liu *et al.*, "Repower: An llm-driven autonomous platform for power system data-guided research," *Patterns*, vol. 6, no. 4, 2025.

[25] X. Chen *et al.*, "X-gridagent: An llm-powered agentic ai system for assisting power grid analysis," *arXiv preprint arXiv:2512.20789*, 2025.

[26] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[27] Q. Xu *et al.*, "Enhancing tool retrieval with iterative feedback from large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 9609–9619.

[28] Z. Z. Wang *et al.*, "Agent workflow memory," *arXiv preprint arXiv:2409.07429*, 2024.

[29] X. Tan *et al.*, "Meta-agent-workflow: Streamlining tool usage in llms through workflow construction, retrieval, and refinement," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 458–467.

[30] J. Li *et al.*, "Alloy: Generating reusable agent workflows from user demonstration," *arXiv preprint arXiv:2510.10049*, 2025.

[31] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, 2020, pp. 6769–6781.

[32] O. Rubin *et al.*, "Learning to retrieve prompts for in-context learning," in *Proceedings of the 2022 conference of the North American chapter of the association for computational linguistics: human language technologies*, 2022, pp. 2655–2671.

[33] M. Luo *et al.*, "Dr.ICL: Demonstration-retrieved in-context learning," *arXiv preprint arXiv:2305.14128*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14128

[34] Y. Huang and J. X. Huang, "A survey on retrieval-augmented text generation for large language models," *ACM Computing Surveys*, 2024.

[35] S. Gupta *et al.*, "A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions," *arXiv preprint arXiv:2410.12837*, 2024.

[36] O. Ram *et al.*, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.

[37] X. L. Do *et al.*, "Prompt optimization via adversarial in-context learning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 7308–7327.

[38] S. Wu *et al.*, "Avatar: Optimizing llm agents for tool usage via contrastive reasoning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 25 981–26 010, 2024.

[39] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.

[40] Y. Liu *et al.*, "Toolgate: Contract-grounded and verified tool execution for llms," *arXiv preprint arXiv:2601.04688*, 2026.

[41] H. Wang *et al.*, "Pro2guard: Proactive runtime enforcement of llm agent safety via probabilistic model checking," *arXiv preprint arXiv:2508.00500*, 2025.

[42] H. Wang *et al.*, "Agentspec: Customizable runtime enforcement for safe and reliable llm agents," *arXiv preprint arXiv:2503.18666*, 2025.

[43] A. Pandey *et al.*, "Robust power flow and three-phase power flow analyses," *IEEE Transactions on Power Systems*, vol. 34, no. 1, pp. 616–626, 2018.

[44] E. O. Badmus and A. Pandey, "Anoca: Ac network-aware optimal curtailment approach for dynamic hosting capacity," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 5338–5345.

[45] M. Z. Liu *et al.*, "Using opf-based operating envelopes to facilitate residential der services," *IEEE Transactions on Smart Grid*, vol. 13, no. 6, pp. 4494–4504, 2022.

[46] H. Moring and J. L. Mathieu, "Inexactness of second order cone relaxations for calculating operating envelopes," in *2023 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*. IEEE, 2023, pp. 1–6.

[47] Y. Yi and G. Verbič, "Fair operating envelopes under uncertainty using chance constrained optimal power flow," *Electric Power Systems Research*, vol. 213, p. 108465, 2022.

[48] E. Foster *et al.*, "Three-phase infeasibility analysis for distribution grid studies," *Electric Power Systems Research*, vol. 212, p. 108486, 2022.

[49] B. Panthee and A. Pandey, "Solving three-phase ac infeasibility analysis to near-zero optimality gap," *arXiv preprint arXiv:2508.15937*, 2025.

[50] "Langchain agents documentation," https://docs.langchain.com/oss/python/langchain/agents, accessed 2026-01-26.

[51] "Crewai concepts: Agents, crews, and flows," CrewAI documentation, accessed 2026-01-26. [Online]. Available: https://docs.crewai.com/en/concepts/agents

[52] "Gpt-4o mini," OpenAI API Documentation, accessed 2026-01-26. [Online]. Available: https://platform.openai.com/docs/models/gpt-4o-mini

[53] "Gpt-5.2," OpenAI API Documentation, accessed 2026-01-26. [Online]. Available: https://platform.openai.com/docs/models/gpt-5.2

[54] "Models," OpenAI API Documentation, accessed 2026-01-26. [Online]. Available: https://platform.openai.com/docs/models

[55] "Qwen/qwen3-14b model card," Hugging Face, accessed 2026-01-26. [Online]. Available: https://huggingface.co/Qwen/Qwen3-14B

[56] "Qwen/qwen3-14b model card," Hugging Face, accessed 2026-01-26. [Online]. Available: https://huggingface.co/Qwen/Qwen3-14B

[57] "gpt-oss-120b & gpt-oss-20b model card," OpenAI, accessed 2026-01-26. [Online]. Available: https://openai.com/index/gpt-oss-model-card/

[58] "Openai-compatible server," vLLM Documentation, accessed 2026-01-26. [Online]. Available: https://docs.vllm.ai/en/stable/serving/openai_compatible_server/

[59] "Nvidia h100 tensor core gpu," NVIDIA Product Page, accessed 2026-01-26. [Online]. Available: https://www.nvidia.com/en-us/data-center/h100/

[60] OpenAI, "text-embedding-3-large (model documentation)," https://platform.openai.com/docs/models/text-embedding-3-large, 2026, accessed: Jan. 2026.

[61] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.